PROJECT REPORT

CANCER CLASSIFICATION AND PREDICTION USING NEURAL NETWORKS

Breast Cancer Wisconsin (Diagnostic) Dataset

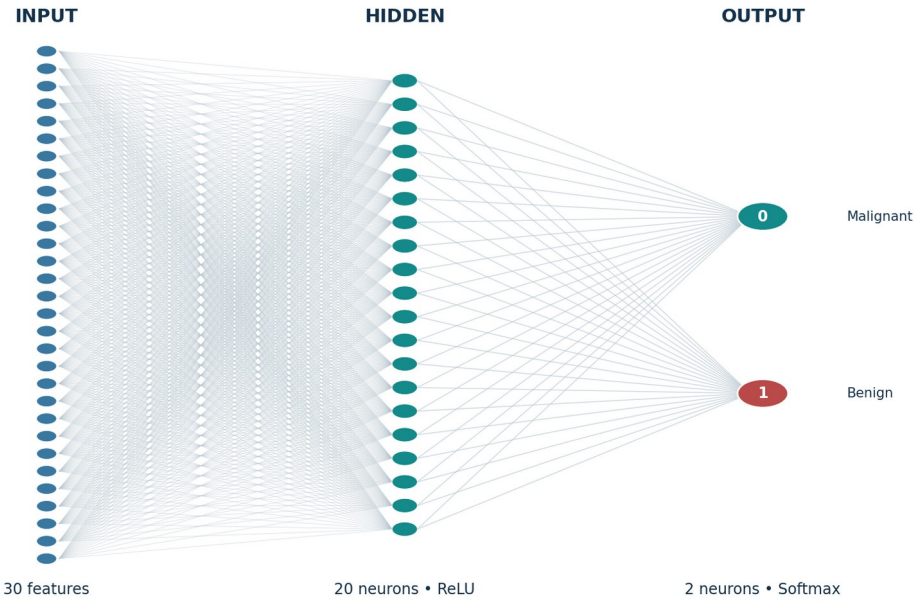


Figure: Conceptual architecture of the Artificial Neural Network used in this project.

Submitted by

SANKET KISHOR SUTAR

CERTIFICATE

Project authorship certificate

This certificate records that the project report titled “Cancer Classification and Prediction Using Neural Networks” has been prepared by SANKET KISHOR SUTAR. The report documents an educational machine-learning implementation using the Breast Cancer Wisconsin (Diagnostic) dataset and describes its data exploration, preprocessing, Artificial Neural Network development, training, evaluation, interactive prediction workflow and model persistence.

This is an author-issued project statement. It records authorship and the scope of the work only; it does not represent approval, endorsement or professional certification by any external person or organization. Results and figures are identified according to their actual source and should be interpreted within the limitations described in this report.

AUTHOR DECLARATION

Project authorship statement

I, SANKET KISHOR SUTAR, confirm that this report describes the Cancer Classification and Prediction Using Neural Networks project and identifies the dataset and software sources used. The technical methods, figures and results are described with their scope and limitations.

The project uses the Breast Cancer Wisconsin (Diagnostic) dataset available through Scikit-learn. The documented implementation uses Python, Pandas, NumPy, Matplotlib, Seaborn, Scikit-learn, TensorFlow/Keras and Gradio in Google Colab. The model maps class 0 to malignant and class 1 to benign.

I understand that a machine-learning model trained and evaluated on a public educational dataset is not a clinically validated diagnostic instrument. Its outputs must not be used for medical decisions, treatment recommendations or patient management.

The numerical results and output screenshots in this revised report are taken from the Google Colab screenshots supplied by SANKET KISHOR SUTAR. The documented run completed 10 epochs and achieved 95.60% training accuracy, 95.65% validation accuracy and 93.86% held-out testing accuracy. The report distinguishes the supplied notebook outputs from explanatory diagrams and tables calculated from the supplied confusion-matrix counts.

I acknowledge all external datasets, documentation and technical references used in this report. Sources are listed in the References section.

Author: SANKET KISHOR SUTAR

ACKNOWLEDGEMENTS

I acknowledge the maintainers of the open-source libraries used in this project, including Scikit-learn, TensorFlow/Keras, NumPy, Pandas, Matplotlib, Seaborn and Gradio. Their documentation and software tools support the data analysis, neural-network development, evaluation and interface workflow described in this report.

I acknowledge the UCI Machine Learning Repository and the Scikit-learn project for providing access to the Breast Cancer Wisconsin (Diagnostic) dataset and its documented loader.

I also acknowledge the maintainers of the Python scientific-computing ecosystem and the authors of the technical references listed at the end of this report.

This report was assembled to document the dataset, model design, workflow, measured outputs and limitations in a clear and reproducible form.

SANKET KISHOR SUTAR

ABSTRACT

This project presents an educational Artificial Intelligence and Machine Learning application for classifying breast tumor samples as malignant or benign using an Artificial Neural Network (ANN). The implementation uses the Breast Cancer Wisconsin (Diagnostic) dataset, distributed through Scikit-learn. The dataset contains 569 samples, 30 real-valued input features and two target classes. The features describe characteristics of cell nuclei measured from digitized images of fine needle aspirates of breast masses [1, 2].

The workflow includes dataset loading, exploratory data analysis, data-quality checks, feature-target separation, stratified training/validation/testing splits, feature standardization, neural-network construction, model training, evaluation and interactive prediction. The implemented network is a feed-forward Sequential model with 30 input features, one hidden Dense layer containing 20 neurons with ReLU activation, and a two-neuron Softmax output layer. The model is configured with the Adam optimizer, a learning rate of 0.001, sparse categorical cross-entropy loss and accuracy as a training metric.

For the final training arrangement, the dataset is first divided into 80% training and 20% testing data using stratification. The 455-sample training portion is further divided into 409 training samples and 46 validation samples. The final StandardScaler is fitted only on the 409 training samples and then applied to validation and test data, avoiding information leakage from those subsets.

An interactive application is developed using Gradio. It accepts 30 numerical measurements in three groups - mean, standard-error and worst measurements - applies the saved scaler and returns the model's predicted class and class probabilities. The application is intended only for educational demonstration.

Keywords: Artificial Neural Network, Breast Cancer Wisconsin Diagnostic Dataset, binary classification, TensorFlow, Keras, StandardScaler, Gradio, machine learning.

TABLE OF CONTENTS

Section	Page
Certificate	2
Declaration	3
Acknowledgements	4
Abstract	5
List of Figures, Tables and Abbreviations	7
Chapter 1 - Introduction	9
Problem Statement, Motivation and Objectives	10
Scope, Applications and Expected Outcomes	11
Chapter 2 - Literature Review	12
Dataset Description and Provenance	13
Dataset Features	14
Chapter 3 - Methodology	15
Exploratory Data Analysis and Data Quality	16
Exploratory Visualizations	17
Correlation Analysis and Heatmap	18-19
Data Preprocessing	20
Neural Network Architecture	21
Model Training	22
Evaluation Metrics	23
System Design	24
Implementation Details	25
Interactive Prediction Application	27
Results and Observations	29-31
Confusion Matrix and Error Analysis	32-33
Testing and Verification	34
Limitations, Ethics and Responsible Use	35
Conclusion and Future Scope	36
References	37
Appendix A - Feature Inventory	38
Appendix B - Project Release and Reproducibility Notes	39

LIST OF FIGURES, TABLES AND ABBREVIATIONS

List of Figures

Figure	Description
Figure 1	End-to-end project workflow
Figure 2	Actual Colab class distribution chart
Figure 3	Actual Colab histograms of selected mean features
Figure 4	Actual Colab box plots of selected measurements
Figure 5	Pearson correlation heatmap of 30 input features
Figure 6	Organization of the 30 input measurements into three groups
Figure 7	Artificial Neural Network architecture
Figure 8	High-level system architecture
Figure 9	Actual Gradio application: input form, prediction control and displayed output
Figure 10	Google Colab DataFrame information and data-quality output
Figure 11	Actual ten-epoch Colab training log
Figure 12	Actual training and validation accuracy graph
Figure 13	Actual training and validation loss graph
Figure 14	Actual held-out test confusion matrix
Figure 15	Actual final evaluation summary
Figure 16	Actual-versus-predicted output table
Figure 17	Actual Keras model summary output

List of Tables

Table	Description
Table 1	Dataset summary
Table 2	Target class mapping
Table 3	Feature groups
Table 4	Training / validation / test allocation
Table 5	Neural-network configuration
Table 6	Evaluation metric definitions
Table 7	Functional test cases
Table 8	Final Colab-run evaluation metrics
Table 9	Class-wise classification report calculated from supplied confusion-matrix counts

Abbreviations

Abbreviation	Meaning
AI	Artificial Intelligence
ANN	Artificial Neural Network

Abbreviation	Meaning
EDA	Exploratory Data Analysis
FNA	Fine Needle Aspirate
ML	Machine Learning
ReLU	Rectified Linear Unit
TP / TN	True Positive / True Negative
FP / FN	False Positive / False Negative
API	Application Programming Interface
UI	User Interface

CHAPTER 1 - INTRODUCTION

1.1 Background

Artificial Intelligence (AI) and Machine Learning (ML) are used to identify patterns in data and support a range of analytical tasks. Classification is a supervised-learning task in which a model learns from labelled examples and predicts a category for new observations. In a healthcare-related educational setting, classification methods can be demonstrated using structured measurements associated with medical samples.

This project focuses on a binary classification task using the Breast Cancer Wisconsin (Diagnostic) dataset. Each row represents a sample described by 30 numerical features. The target identifies the sample as malignant or benign. A neural network is trained to learn associations between the numerical measurements and the provided target labels.

1.2 Project Overview

The implementation begins by loading the dataset from Scikit-learn. Pandas is used to organize the observations into a DataFrame. Exploratory data analysis is performed to inspect dimensions, missing values, duplicate rows, descriptive statistics, class frequencies, feature distributions and correlations. The data is then separated into input features and target labels, divided into training, validation and test subsets, and standardized.

A Sequential Artificial Neural Network is built using TensorFlow/Keras. The network has an input dimension of 30, one hidden Dense layer of 20 neurons using ReLU, and an output Dense layer of two neurons using Softmax. The trained model is evaluated on a held-out test set, and an interactive Gradio application is created to demonstrate predictions from user-provided numerical measurements.

1.3 Nature of the Project

The project is an educational prototype intended to demonstrate data analysis, neural-network development, evaluation and interface integration. It does not use live hospital records, prospective clinical data or a medically approved workflow. The dataset labels are used as the target for supervised learning; model output is not a clinical diagnosis.

IMPORTANT DISTINCTION

The model learns from a public, labelled dataset. It does not examine medical images directly and does not independently confirm cancer. It classifies numerical feature vectors supplied to the model.

PROBLEM STATEMENT, MOTIVATION AND OBJECTIVES

1.4 Problem Statement

The project addresses the supervised-learning problem of predicting whether a sample in the Breast Cancer Wisconsin (Diagnostic) dataset belongs to the malignant or benign class using 30 numerical measurements. The central technical problem is to construct a neural-network classifier that can learn from labelled examples, generalize to unseen test samples and expose its output through a simple interface.

1.5 Motivation

The dataset provides a compact, well-structured example for understanding a complete machine-learning workflow. It allows a author to practice data exploration, feature scaling, stratified sampling, neural-network architecture design, training-curve interpretation and classification metrics. It also provides an opportunity to examine why recall and false-negative analysis matter in a high-stakes domain, even when the project itself is only an educational exercise.

1.6 Objectives

- Load and understand the Breast Cancer Wisconsin (Diagnostic) dataset.
- Examine the dataset for missing values, duplicates and distribution characteristics.
- Separate the 30 numerical input features from the binary target variable.
- Create stratified training, validation and testing subsets.
- Standardize features using parameters estimated from training data only.
- Develop and train an Artificial Neural Network using TensorFlow/Keras.
- Evaluate the model using accuracy, precision, recall, F1-score and confusion matrix.
- Create an interactive Gradio interface that accepts 30 numerical values and displays model output.
- Save the trained model and preprocessing artefacts for later reuse.
- Document the implementation, observations, limitations and future improvements.

1.7 Expected Outcome

The expected deliverable is a reproducible educational classification prototype with a saved neural network, fitted scaler, feature-order metadata, evaluation artefacts, an interactive demonstration interface and a documented workflow. The project brief sets an accuracy above 90% as an intended benchmark; actual performance must be established from a documented run.

SCOPE, APPLICATIONS AND EXPECTED OUTCOMES

1.8 Scope

The scope is restricted to the Breast Cancer Wisconsin (Diagnostic) dataset provided by Scikit-learn and a two-class supervised-learning task. The implementation covers loading, exploratory analysis, preprocessing, a feed-forward ANN, training and validation, test evaluation, an interactive numeric-input demonstration and saving the trained artefacts.

1.9 Applications of the Underlying Concepts

The general techniques demonstrated in this project - tabular data preparation, feature scaling, binary classification, neural-network optimization and evaluation - are relevant to many data-analysis contexts. The project manual identifies medical diagnosis support systems, clinical decision-making, healthcare analytics, disease-prediction systems and research and development as application areas. These are broad application categories; this author prototype is not approved for use in any of those operational settings.

1.10 Project Boundaries

- The model accepts numerical feature values, not a raw mammogram, biopsy image or free-text clinical report.
- The dataset is a fixed public benchmark with 569 samples and 30 features.
- The target labels are inherited from the source dataset; the project does not independently establish ground truth.
- The application is a demonstration interface, not a deployed clinical service.
- No patient identity, hospital integration, treatment advice or medical-record storage is included.
- The project does not include external validation on an independent cohort or prospective population.

1.11 Expected Learning Outcomes

The project is intended to develop understanding of the full supervised-learning pipeline and of how model outputs should be interpreted. The author should be able to explain the data source, all 30 features, the reason for stratification and scaling, the neural-network layers, the metrics and the limitations of the results.

CHAPTER 2 - LITERATURE REVIEW

2.1 Dataset Research Background

The Breast Cancer Wisconsin (Diagnostic) dataset is documented by the UCI Machine Learning Repository. It contains 569 instances and 30 continuous features. The features are computed from digitized images of fine needle aspirates of breast masses and describe characteristics of cell nuclei. UCI identifies the associated task as classification and the subject area as health and medicine [1].

2.2 Supervised Classification

Supervised classification uses labelled training examples. During training, a model receives input features together with known target labels. It adjusts internal parameters to reduce a loss function. Once training is complete, the model can produce a predicted class for a new input. The separation of training and test data is important because evaluating on samples used during learning can provide an overly optimistic estimate of generalization.

2.3 Neural Networks for Tabular Classification

A feed-forward neural network consists of layers of computational units. Dense layers connect each unit in one layer to all units in the next layer. Non-linear activation functions allow the network to learn relationships beyond a simple linear mapping. In this project, a small network is selected to keep the architecture understandable and suitable for a author demonstration: one hidden layer with 20 neurons and an output layer with two neurons.

2.4 Feature Standardization

Standardization transforms each feature by subtracting a mean and dividing by a standard deviation. Scikit-learn's StandardScaler estimates these quantities from the data supplied to fit and reuses them when transform is called on later data [3]. Fitting only on the training subset prevents validation or test observations from influencing the transformation parameters.

2.5 Model Evaluation

Accuracy summarizes the fraction of correct predictions, but class-wise precision, recall and F1-score provide additional information. A confusion matrix shows the counts of correct and incorrect predictions by actual and predicted class. In a medical-themed classification exercise, false negatives should be examined explicitly because they represent malignant-labelled samples predicted as benign. Metric definitions and averaging choices should be stated clearly [5, 6].

2.6 Position of This Project

This project is an educational implementation of a neural-network classifier using a public dataset and standard open-source libraries. It does not propose a novel medical method or claim superiority over clinical systems. Its contribution is the integration of data preparation, ANN training, evaluation and a small interactive demonstration into a documented project workflow.

DATASET DESCRIPTION AND PROVENANCE

2.7 Dataset Source

The selected dataset is the Breast Cancer Wisconsin (Diagnostic) dataset from the UCI Machine Learning Repository, accessed in the project through Scikit-learn's load_breast_cancer() dataset loader. Scikit-learn documents 569 total samples, 30 input dimensions and two target classes: 212 malignant and 357 benign [1, 2].

The UCI description states that the features are calculated from digitized images of fine needle aspirates of breast masses and describe characteristics of cell nuclei [1]. This provenance is important: the table does not consist of 569 raw images. It contains precomputed numerical features derived from image analysis.

2.8 Dataset Summary

Property	Value
Dataset name	Breast Cancer Wisconsin (Diagnostic)
Repository	UCI Machine Learning Repository
Project access method	sklearn.datasets.load_breast_cancer()
Total samples	569
Input features	30
Target classes	2
Malignant samples	212 (37.26%)
Benign samples	357 (62.74%)
Feature type	Real-valued numerical measurements
Task	Binary classification

2.9 Target Labels

Encoded value	Dataset class	Interpretation
0	malignant	Malignant class
1	benign	Benign class

2.10 Dataset Suitability

The dataset is suitable for demonstrating tabular binary classification because it has a defined target, numeric features and a moderate number of observations. Its class frequencies are not equal, so stratified sampling is used to preserve approximate class proportions across data subsets. The dataset's modest size also means that results should be interpreted cautiously and not generalized to a clinical population without external validation.

DATASET FEATURES AND ORGANIZATION

2.11 Three Feature Groups

The 30 input features are organized into three groups of ten measurements. The groups describe the mean value, standard error and worst (largest or most extreme) value of ten characteristics. This structure is reflected in the feature names and is also used to group inputs in the Gradio interface.

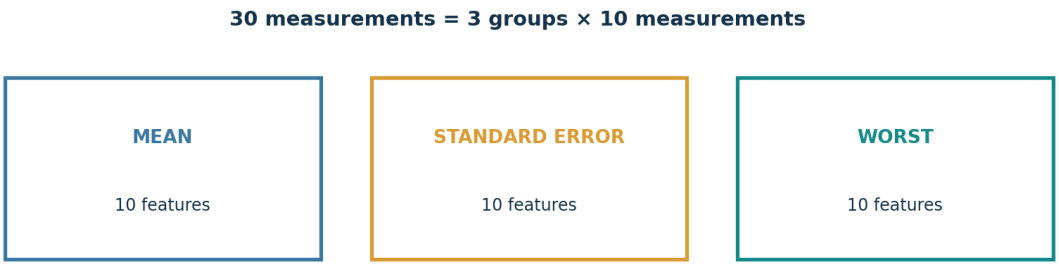


Figure 6. Organization of the 30 input measurements into three groups.

Measurement family	Features included
Size and geometry	radius, perimeter, area
Surface and texture	texture, smoothness
Shape characteristics	compactness, concavity, concave points, symmetry, fractal dimension

2.12 Meaning of Representative Features

- Radius: a measure associated with distances from the centre to points on the perimeter.
- Texture: variation in grayscale values.
- Perimeter: measurement of the boundary length.
- Area: measurement of the enclosed region.
- Smoothness: local variation in radius lengths.
- Compactness: a boundary/area-derived shape descriptor.
- Concavity: degree of concave portions of the contour.
- Concave points: number or extent of concave portions of the contour.
- Symmetry: measure of symmetry in the nucleus boundary.
- Fractal dimension: a shape descriptor based on a coastline-approximation concept.

2.13 Feature Naming

Each measurement is repeated in the mean, standard-error and worst groups. Examples include mean radius, radius error and worst radius. The original 30-column order is preserved through training, evaluation and application input handling. The feature names are saved with the model artefacts to reduce the risk of incorrect column order when new measurements are supplied.

CHAPTER 3 - METHODOLOGY

3.1 Overall Methodology

The project follows a sequential supervised-learning workflow. It begins with dataset collection and exploration, continues through data preparation and neural-network training, and concludes with independent testing, an interactive prediction interface and model persistence.



Figure 1. End-to-end workflow followed by the project.

3.2 Project Phases

Phase	Main activities	Output
1. Data collection	Load the dataset and identify its fields.	569 × 30 feature matrix and target vector
2. Data exploration	Inspect quality, distributions and correlations.	EDA tables and figures
3. Preprocessing	Separate X/y, split data and standardize features.	Scaled train, validation and test matrices
4. Model development	Define ANN layers and compile the model.	Compiled Keras model
5. Model training	Train over epochs and monitor validation metrics.	Trained model and history
6. Evaluation	Measure test performance and inspect errors.	Metrics and confusion matrix
7. Prediction	Connect numeric inputs to model and scaler.	Interactive Gradio demonstration
8. Persistence	Save model, scaler, metadata and results.	Reusable project artefacts

3.3 Reproducibility

The data splits use random_state=42 and stratify labels. The training configuration is recorded in the metadata. Neural-network weight initialization and training can still vary unless deterministic settings are applied consistently. The final project report should describe the actual run from which its metrics were obtained and should not mix results from different runs.

EXPLORATORY DATA ANALYSIS AND DATA QUALITY

3.4 Initial Inspection

The dataset is loaded through load_breast_cancer() and converted into a Pandas DataFrame. The 30 input columns are taken from the dataset's feature_names field and the target column is added separately. The resulting frame has 569 rows and 31 columns: 30 features plus one target.

3.5 Missing-Value Check

The project checks missingness column by column using DataFrame.isnull().sum(). The canonical Scikit-learn dataset used for the descriptive analysis in this report contains zero missing values across its 569 × 31 frame [2]. No imputation is required for that source frame.

3.6 Duplicate-Record Check

Exact duplicate rows are examined using DataFrame.duplicated().sum(). The canonical dataset frame contains zero exact duplicate rows when the 30 features and target column are considered together. A duplicate check is still retained in the project workflow because data cleaning is an explicit part of the assignment brief.

3.7 Descriptive Statistics

Pandas describe() is used to summarize count, mean, standard deviation, minimum, quartiles and maximum for numerical columns. These summaries help reveal scale differences and broad variation across measurements. The values are not used as clinical reference ranges; they describe only this dataset.

Feature	Mean	Std. dev.	Minimum	Maximum
mean radius	14.127	3.524	6.981	28.110
mean texture	19.290	4.301	9.710	39.280
mean perimeter	91.969	24.299	43.790	188.500
mean area	654.889	351.914	143.500	2501.000

The statistics above are dataset-level descriptive values calculated from the canonical Scikit-learn dataset. They should not be interpreted as diagnostic cut-offs.

EXPLORATORY VISUALIZATIONS

3.8 Target-Class Distribution

The dataset contains 212 malignant samples and 357 benign samples. These correspond to approximately 37.26% malignant and 62.74% benign. The class distribution is therefore not equal. The project uses stratified sampling for the train/test split and for the validation split so that the proportions remain approximately represented in each subset.

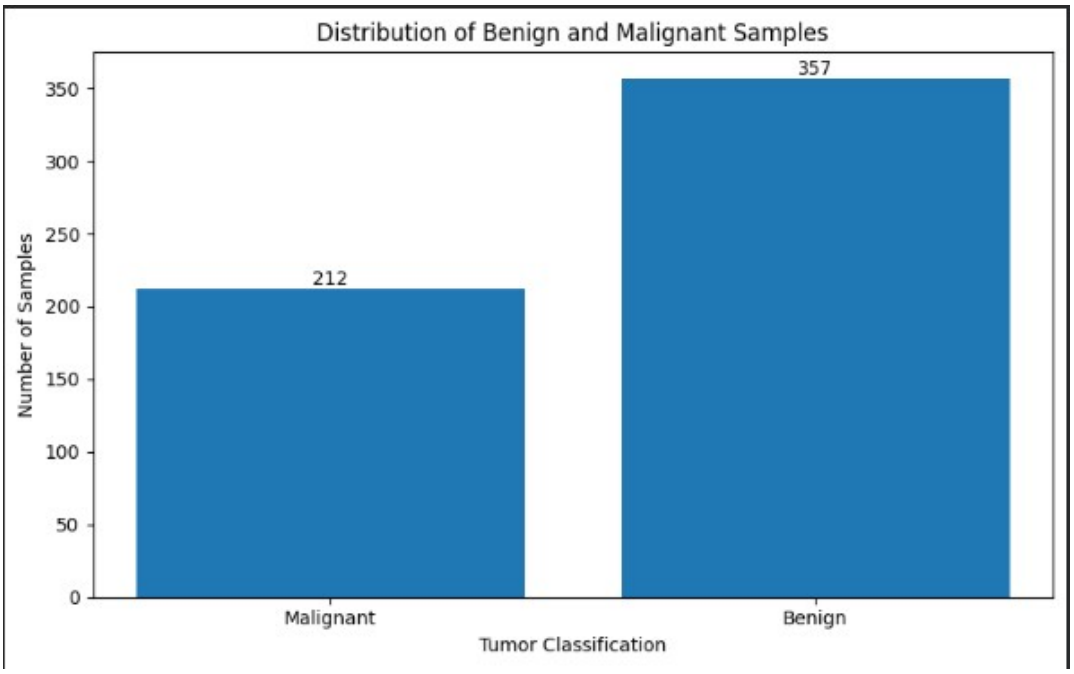


Figure 2. Actual Google Colab bar chart showing 212 malignant and 357 benign samples.

3.9 Feature Histograms

Histograms are used to inspect the distributions of selected mean measurements: radius, texture, perimeter and area. The plots distinguish malignant and benign samples so that distributional differences and overlap can be observed. Histograms are descriptive; they do not prove that a feature alone can classify every sample correctly.

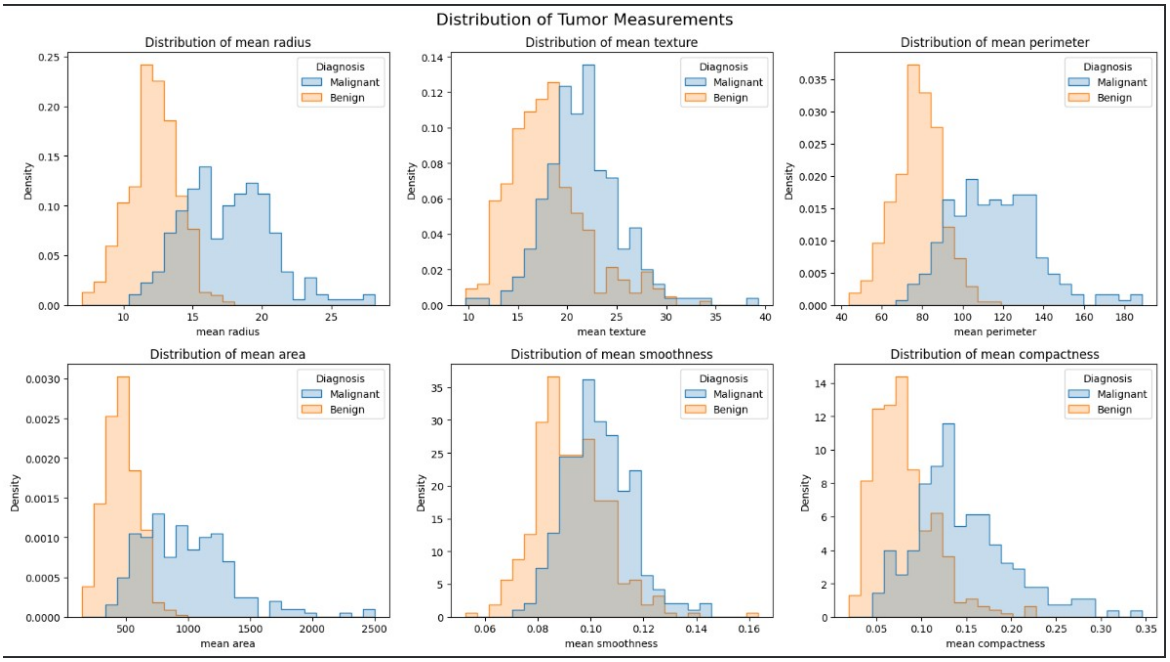


Figure 3. Actual Google Colab histograms for six selected mean tumor measurements, separated by diagnosis.

FEATURE COMPARISON AND CORRELATION ANALYSIS

3.10 Box-Plot Analysis

Box plots are used to compare the distributions of selected features across the two classes. The median, interquartile range, whiskers and possible outlying observations can be compared visually. An observation outside a whisker is not automatically an error; it may be a valid sample. Any record-removal decision should be based on data provenance and justified evidence.

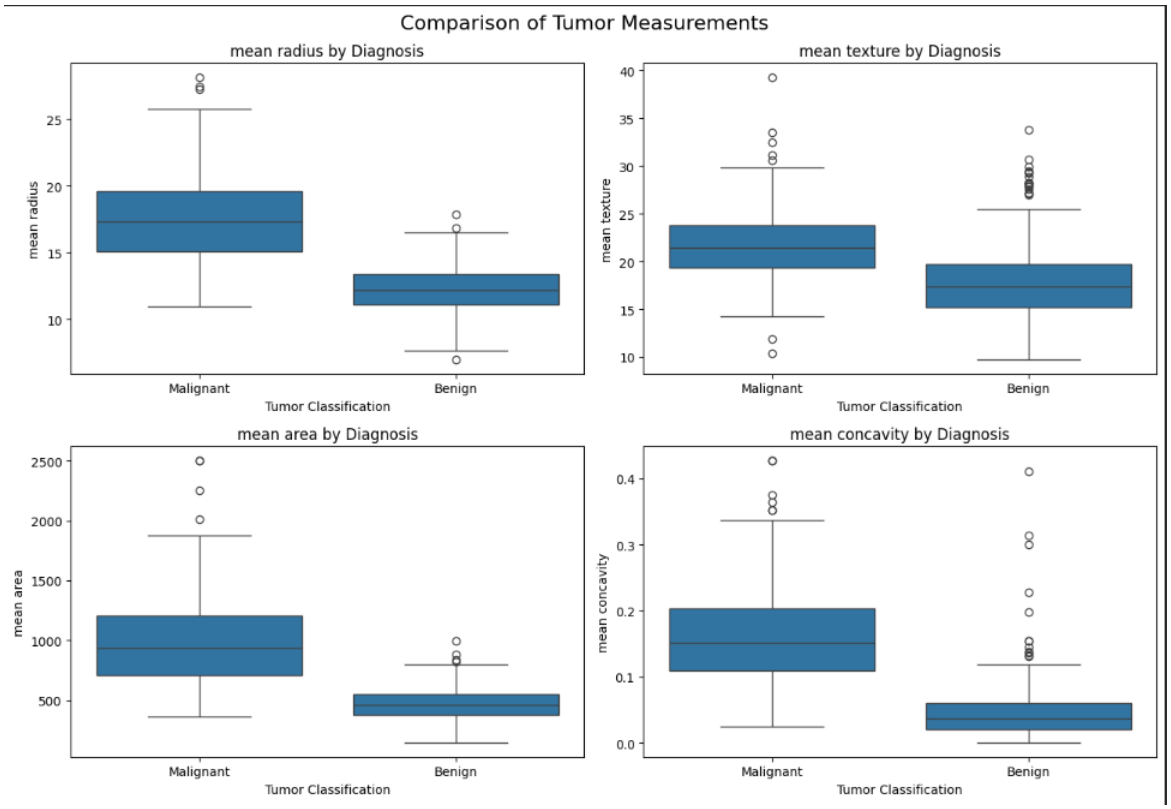


Figure 4. Actual Google Colab box plots comparing selected tumor measurements between malignant and benign samples.

3.11 Correlation Analysis

A Pearson correlation matrix is calculated for the 30 input features. Correlation values range from -1 to +1 and describe linear association. Highly correlated features may carry overlapping information. Correlation does not establish causation and does not, by itself, justify deleting a feature. The heatmap is used as an exploratory aid rather than an automatic feature-selection rule.

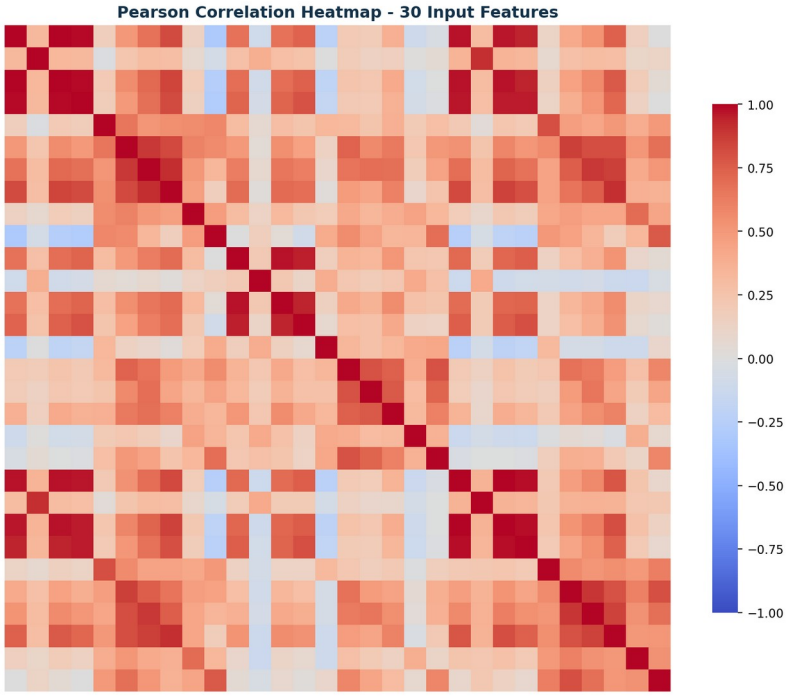


Figure 5. Pearson correlation heatmap for the 30 input features, generated from the project DataFrame.

DATA PREPROCESSING

3.12 Separation of Features and Target

The independent-variable matrix X contains the 30 numerical measurements. The dependent-variable vector y contains the target labels. The encoding used by Scikit-learn is 0 for malignant and 1 for benign [2]. Because the labels are already integers, additional label encoding is not required for the selected sparse categorical cross-entropy setup.

3.13 Stratified Train-Test Split

The complete dataset is divided using train_test_split with test_size=0.20, random_state=42 and stratify=y. This produces 455 initial training samples and 114 test samples. The test subset is kept aside for the final evaluation.

3.14 Training-Validation Split

The 455-sample training portion is further divided using test_size=0.10, random_state=42 and stratify=y_train. This produces 409 model-fitting samples and 46 validation samples. Validation samples are used to monitor generalization during training and are not used for gradient updates.

Subset	Total samples	Malignant (0)	Benign (1)
Model-fitting training	409	153	256
Validation	46	17	29
Final testing	114	42	72

3.15 Standardization and Leakage Control

StandardScaler applies z-score standardization: z = (x - mean) / standard deviation. For the final training pipeline, the scaler is fitted only on the 409 model-fitting samples. The same fitted scaler is then used to transform the 46 validation samples and 114 test samples. The same saved scaler is used by the Gradio prediction function. This prevents validation/test information from influencing the fitted scaling parameters [3].

IMPLEMENTATION CONSISTENCY

The earlier exploratory preprocessing cell may create an initial scaler on the 455-sample training portion. The final model pipeline uses the later training_scaler fitted on the 409-sample fitting subset. The saved scaler and deployment code must use this final training_scaler.

NEURAL NETWORK ARCHITECTURE

3.16 Artificial Neural Network

The project uses a feed-forward Artificial Neural Network implemented as a TensorFlow/Keras Sequential model. It accepts a vector of 30 standardized measurements, processes the values through a fully connected hidden layer, and returns probabilities for two classes.

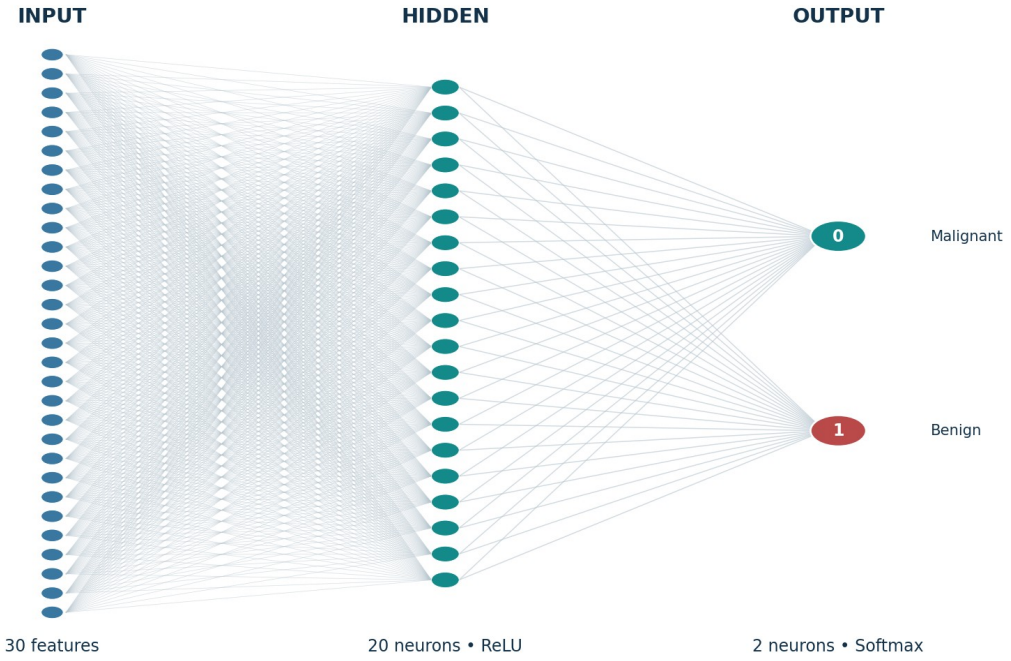


Figure 7. ANN architecture: 30 inputs, 20 hidden neurons and 2 output neurons.

Layer	Configuration	Role
Input	30 features	Receives standardized measurement vector
Hidden Dense	20 neurons, ReLU	Learns non-linear feature combinations
Output Dense	2 neurons, Softmax	Returns class probabilities

3.17 Trainable Parameters

The hidden layer has (30 × 20) weights plus 20 biases, giving 620 parameters. The output layer has (20 × 2) weights plus 2 biases, giving 42 parameters. The total is 662 trainable parameters.

Hidden layer: (30 × 20) + 20 = 620. Output layer: (20 × 2) + 2 = 42. Total parameters: 620 + 42 = 662.

3.18 Activation Functions

ReLU is used in the hidden layer: f(x) = max(0, x). It introduces non-linearity and returns zero for negative inputs while passing positive inputs through. Softmax is used in the two-neuron output layer to produce a normalized pair of class probabilities that sum to one.

3.19 Loss and Optimization

The network is compiled with the Adam optimizer at a learning rate of 0.001 and sparse categorical cross-entropy loss. Sparse categorical cross-entropy is appropriate for integer-encoded class labels with a two-neuron Softmax output. Accuracy is used as a training and validation metric.

MODEL TRAINING

3.20 Training Configuration

Parameter	Configuration
Framework	TensorFlow / Keras
Model type	Sequential feed-forward ANN
Input features	30
Hidden layer	20 Dense neurons
Hidden activation	ReLU
Output layer	2 Dense neurons
Output activation	Softmax
Optimizer	Adam
Learning rate	0.001
Loss	Sparse Categorical Crossentropy
Metric	Accuracy
Epochs	10
Batch size	32
Training samples	409
Validation samples	46

3.21 Training Process

The model is trained using model.fit() with the standardized 409-sample training matrix and its integer labels. The validation dataset is passed separately through validation_data. Training samples are shuffled. The history object stores training and validation loss and accuracy after each epoch.

3.22 Interpreting Learning Curves

Training accuracy describes performance on the data used for weight updates. Validation accuracy describes performance on the held-out validation subset. Training loss and validation loss quantify prediction error under the selected loss function. A widening gap between training and validation results may indicate overfitting; low performance on both may indicate underfitting. Curves must be interpreted together rather than using one value in isolation.

MODEL EVALUATION METRICS

3.23 Testing Procedure

After training and validation, the final model is evaluated on the 114-sample test subset that was not used for gradient updates or scaler fitting. The model generates two probabilities for each sample. The predicted class is selected using the index of the largest probability. The actual test labels are compared with predicted labels to calculate the metrics.

3.24 Evaluation Measures

Metric	Definition	Interpretation
Accuracy	(TP + TN) / (TP + TN + FP + FN)	Overall fraction of correct predictions
Precision	TP / (TP + FP)	Reliability of positive-class predictions
Recall / Sensitivity	TP / (TP + FN)	Fraction of actual positive cases detected
F1-score	2 × (Precision × Recall) / (Precision + Recall)	Harmonic mean of precision and recall
Specificity	TN / (TN + FP)	Fraction of actual negative cases correctly identified

3.25 Positive-Class Convention

For the explicitly reported Malignant-class precision, recall and F1-score, malignant (label 0) is treated as the positive class. This convention must be stated because the dataset's encoded label 0 is malignant and label 1 is benign. The classification report also includes per-class metrics for both labels.

3.26 Confusion Matrix

With label order [0, 1], the confusion matrix rows represent actual classes and columns represent predicted classes. The cell at row 0, column 0 is the number of malignant samples correctly classified as malignant (TP for malignant). Row 0, column 1 is malignant samples classified as benign (FN). Row 1, column 0 is benign samples classified as malignant (FP). Row 1, column 1 is benign samples correctly classified as benign (TN).

SAFETY-RELEVANT INTERPRETATION

False negatives are especially important to discuss in a malignant-versus-benign educational classification task. However, even a low false-negative count on this test split does not establish clinical safety or diagnostic performance in real patients.

SYSTEM DESIGN

4.1 High-Level Architecture

The system consists of a user-facing input interface, input validation, a preprocessing component and a trained neural-network model. The Gradio interface collects the measurements in the original feature order. The fitted StandardScaler transforms the values, and the neural network returns class probabilities. The interface displays the predicted class and probabilities with an educational disclaimer.

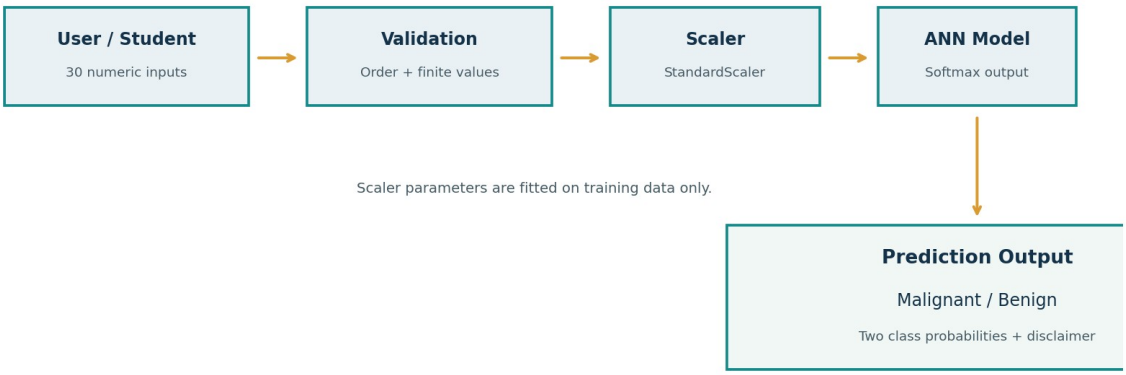


Figure 8. High-level system architecture for interactive prediction.

4.2 Main Components

Component	Responsibility
Input interface	Collect 30 numerical measurements.
Validation layer	Check input count, numeric validity, finiteness and non-negative values.
Feature ordering	Arrange values according to saved feature_names.json.
Preprocessing	Apply the saved training StandardScaler.
ANN inference	Generate two class probabilities using the saved model.
Output display	Show predicted class, probabilities and disclaimer.
Persistence	Store model, scaler, metadata and results in Drive.

4.3 Data Flow

A user supplies 30 measurements. The application checks that all inputs are present and valid. The values are placed into a one-row DataFrame using the saved feature names. The scaler transforms the row using training-set statistics. The model computes Softmax probabilities. The class with the larger probability is displayed as the predicted category. A range warning is shown when any value is outside the observed fitting-data range.

IMPLEMENTATION DETAILS

4.4 Development Environment

Technology	Role in the project
Python	Main programming language
Google Colab	Cloud notebook and execution environment
NumPy	Numerical arrays and calculations
Pandas	Tabular data handling and result tables
Matplotlib	Plots and training curves
Seaborn	Statistical visualizations and heatmaps
Scikit-learn	Dataset loader, train/test split, scaler and metrics
TensorFlow / Keras	Neural-network construction, training and inference
Gradio	Interactive prediction interface

4.5 Dataset Loading and DataFrame Creation

The dataset is loaded with sklearn.datasets.load_breast_cancer(). The feature array is converted to a DataFrame using the provided feature_names. The target array is appended as a target column. The DataFrame is used for exploratory inspection and is retained separately from the transformed arrays.

4.6 Model Construction and Compilation

A Keras Sequential model is constructed with an Input layer of 30 features, a Dense hidden layer of 20 units with ReLU activation and a Dense output layer of two units with Softmax activation. The model is compiled with Adam, learning rate 0.001, sparse categorical cross-entropy and accuracy.

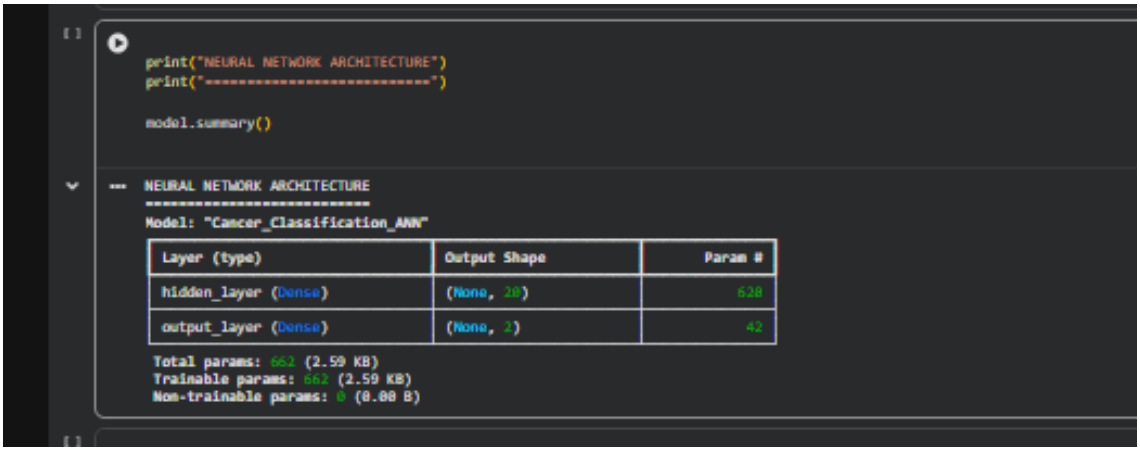


Figure 17. Actual Keras model summary captured in Google Colab, showing 662 trainable parameters.

4.7 Training and Evaluation Code Organization

Training is performed with model.fit() for 10 epochs and batch size 32. The history object is converted into a table and used to plot training and validation accuracy and loss. The final test predictions are generated using model.predict() and converted to integer labels with argmax. Scikit-learn functions calculate accuracy, class-wise precision, recall, F1-score and the confusion matrix.

4.8 Code Quality Practices

- Keep the notebook organized into clearly titled sections and logical cells.
- Use descriptive variable names for training, validation and testing data.

- Preserve the feature order throughout inference.
- Fit the scaler on training data only.
- Keep the test set separate from training and validation decisions.
- Store the trained model and supporting preprocessing metadata together.
- Record actual package versions and result outputs in the final artefacts.

INTERACTIVE PREDICTION APPLICATION

4.9 Gradio Interface

The running prediction interface was implemented with Gradio Blocks. The interface groups the 30 numerical measurements into three sections: mean measurements, standard-error measurements and worst measurements. Figure 9 contains screenshots from the live Google Colab-hosted interface, showing the numerical input form, the prediction control and the returned output. The interface uses the original dataset feature order so that each entered value is passed to the corresponding model input.

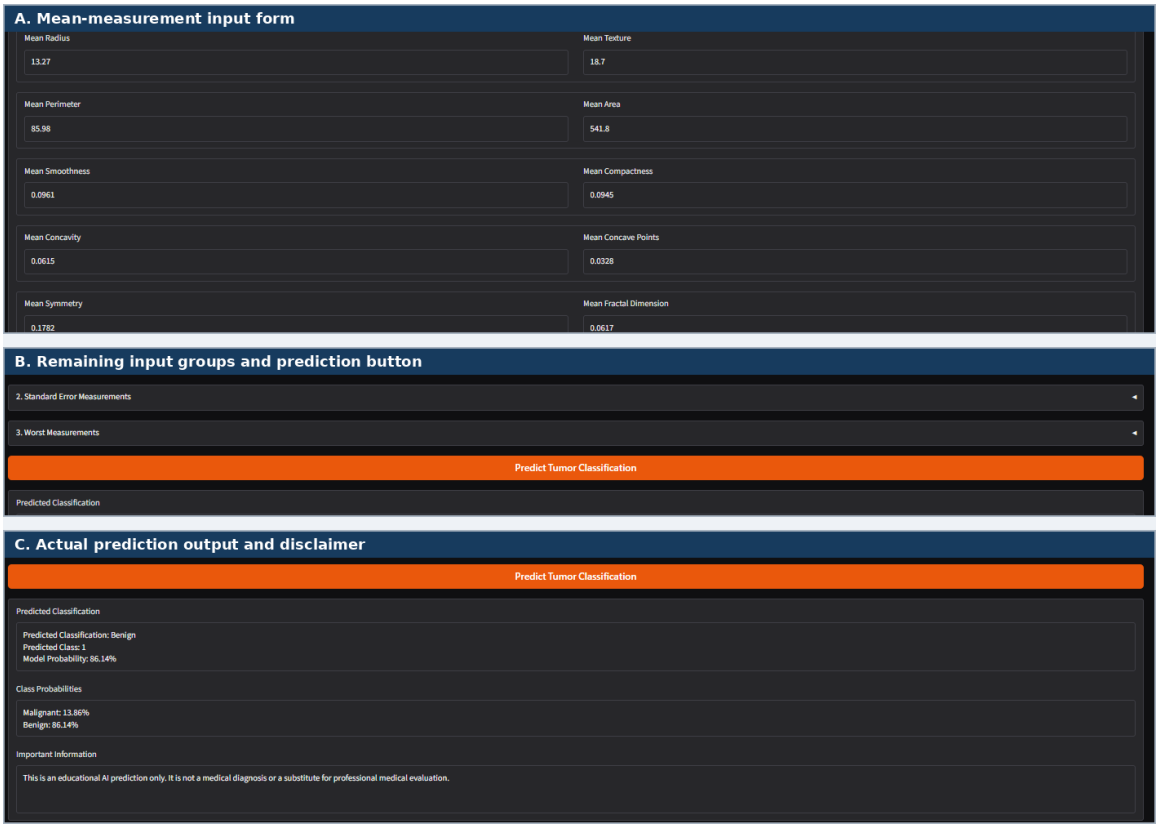


Figure 9. Actual Gradio application screenshots: (A) mean-measurement input form, (B) remaining input groups and prediction control, and (C) displayed prediction output.

4.10 Input Components

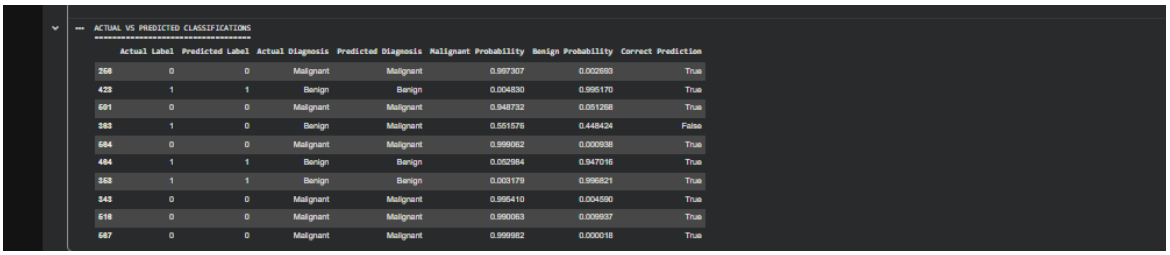
The interface uses numerical input components for each of the 30 features. The feature labels are derived from the saved feature names. The default demonstration values are calculated from the median of the training subset; they are example defaults and do not represent a real patient or a clinical reference profile.

4.11 Prediction Function

The prediction function checks that exactly 30 values are supplied, validates that values are numeric, finite and non-negative, constructs an ordered one-row DataFrame, applies the saved training scaler and sends the standardized vector to the trained model. The output includes the predicted class, the model's two class probabilities and a warning/disclaimer message.

4.12 Probability Display

The Softmax output provides a model probability for malignant (class 0) and benign (class 1). In the supplied interface demonstration shown in Figure 9(C), the entered values produced the displayed result "Benign" (class 1), with a model probability of 86.14%; the displayed malignant probability was 13.86%. This is one demonstration output from the interface, not a held-out test result, a clinically verified case or a calibrated estimate of an individual's disease risk. The interface therefore labels the output as a model probability and includes an educational-use disclaimer.



ACTUAL VS PREDICTED CLASSIFICATIONS

	Actual Label	Predicted Label	Actual Diagnosis	Predicted Diagnosis	Malignant Probability	Benign Probability	Correct Prediction
268	0	0	Malignant	Malignant	0.997307	0.002693	True
423	1	1	Benign	Benign	0.004830	0.995170	True
601	0	0	Malignant	Malignant	0.948732	0.051268	True
383	1	0	Benign	Malignant	0.551576	0.448424	False
684	0	0	Malignant	Malignant	0.999062	0.000938	True
484	1	1	Benign	Benign	0.052984	0.947016	True
353	1	1	Benign	Benign	0.003179	0.996821	True
343	0	0	Malignant	Malignant	0.995410	0.004590	True
618	0	0	Malignant	Malignant	0.990063	0.009937	True
687	0	0	Malignant	Malignant	0.999982	0.000018	True

Figure 16. Actual-versus-predicted output table for the first ten held-out test samples, including class probabilities.

4.13 Application Limitations

The application was demonstrated from a Google Colab-hosted Gradio session. The share URL displayed by Gradio is temporary and is not permanent production hosting. The application must be relaunched while the Colab runtime is active. A durable deployment would require a separate hosting environment, dependency management, security review, input controls and monitoring. The current interface is an educational prototype and must not be used for clinical decision-making.

CHAPTER 5 - RESULTS AND OBSERVATIONS

5.1 Dataset and EDA Results

The Breast Cancer Wisconsin (Diagnostic) dataset contains 569 samples and 30 numerical input features. The target distribution is 212 malignant samples (37.26%) and 357 benign samples (62.74%). The supplied Colab inspection shows a 569-row, 31-column DataFrame with 569 non-null entries in each column. The missing-value check reports zero missing values. The duplicate-check code is part of the workflow; the canonical dataset is also checked for exact duplicate rows before modelling [2].



Figure 10. DataFrame information output from the author's Google Colab session, showing 569 records and 31 columns.

Output provenance: Dataset dimensions, DataFrame inspection, class counts, missing-value output, descriptive statistics, class-distribution chart, selected histograms, box plots, training logs, training-history values, accuracy and loss curves, actual-versus-predicted table, confusion matrix and final evaluation summary are documented using the Google Colab screenshots supplied by SANKET KISHOR SUTAR. The live Gradio form and its displayed Benign prediction output are now included in Figure 9. The full correlation heatmap and architecture/system diagrams are report visualizations based on the dataset and documented model design rather than unaltered notebook screenshots.

5.2 Data Split Results

Subset	Samples	Malignant	Benign
Fitting / training	409	153	256
Validation	46	17	29
Test	114	42	72
Total	569	212	357

5.3 Model Training Results

The supplied Google Colab run completed 10 epochs with a batch size of 32. Final training accuracy was 95.60% and final validation accuracy was 95.65%. Final training loss was 0.1188, while final validation loss was 0.2101. The values for all ten epochs are provided in Table 19 and are illustrated by the supplied training log and learning-curve screenshots.

Epoch	Train accuracy	Train loss	Validation accuracy	Validation loss
1	63.57%	0.6070	73.91%	0.5383
2	84.60%	0.4195	86.96%	0.4387
3	91.93%	0.3057	84.78%	0.3499
4	93.15%	0.2401	89.13%	0.3065
5	94.62%	0.1989	89.13%	0.2799
6	95.11%	0.1715	91.30%	0.2582

Epoch	Train accuracy	Train loss	Validation accuracy	Validation loss
7	95.50%	0.1527	91.30%	0.2428
8	95.60%	0.1386	91.30%	0.2299
9	95.60%	0.1278	95.65%	0.2201
10	95.60%	0.1188	95.65%	0.2101

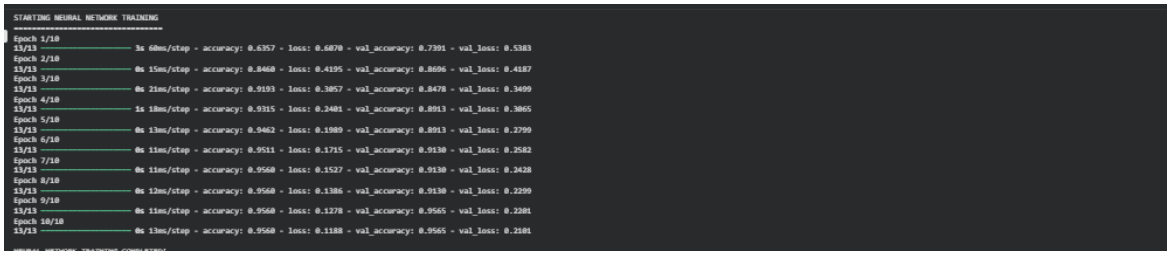


Figure 11. Ten-epoch training log captured from the author's Google Colab session.

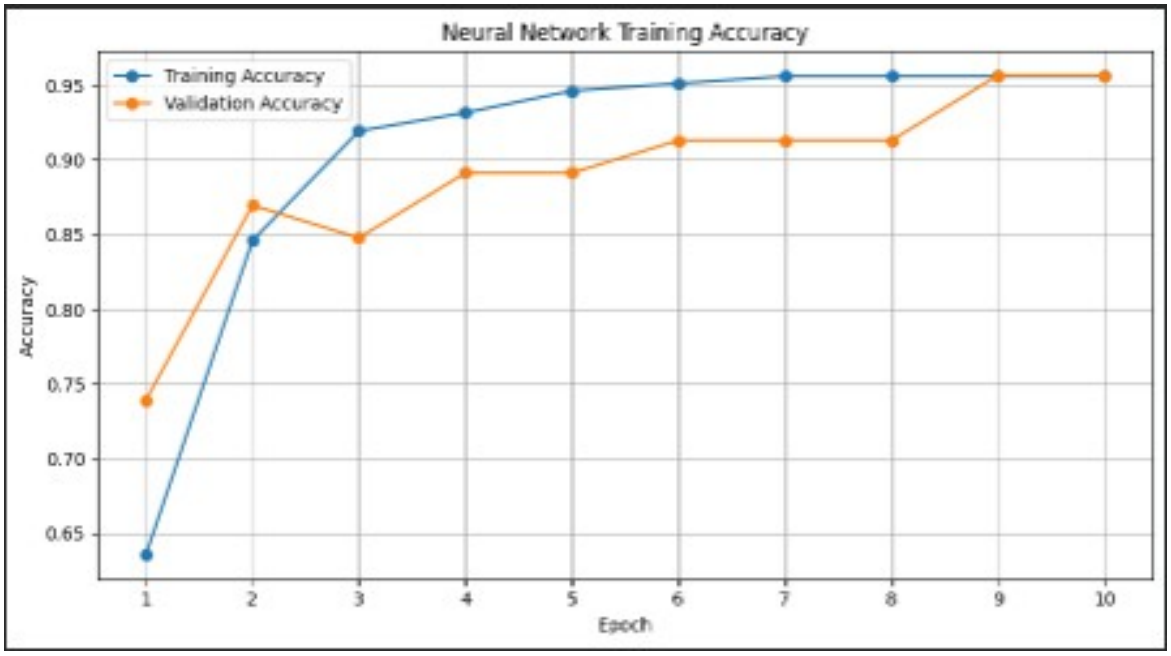


Figure 12. Training and validation accuracy over 10 epochs from the author's Google Colab session.

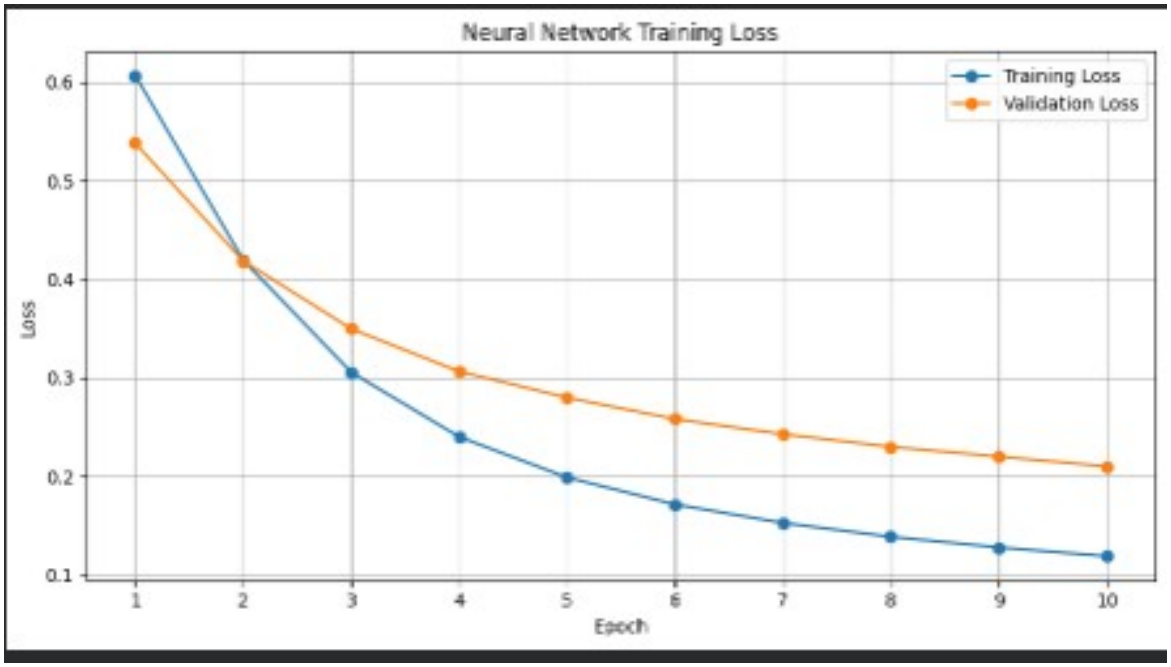


Figure 13. Training and validation loss over 10 epochs from the author's Google Colab session.

5.4 Test Results

On the held-out set of 114 samples, the supplied Colab run achieved 93.86% testing accuracy with a sparse categorical cross-entropy loss of 0.1408. The model correctly classified 107 samples and incorrectly classified 7. Class-wise precision, recall and F1-score are reported separately because overall accuracy alone does not describe the two classes' error patterns.

Performance metric	Colab run result
Testing accuracy	93.86%
Testing loss	0.1408
Malignant precision	92.68%
Malignant recall / sensitivity	90.48%
Malignant F1-score	91.57%
Benign precision	94.52%
Benign recall	95.83%
Benign F1-score	95.17%
Specificity	95.83%

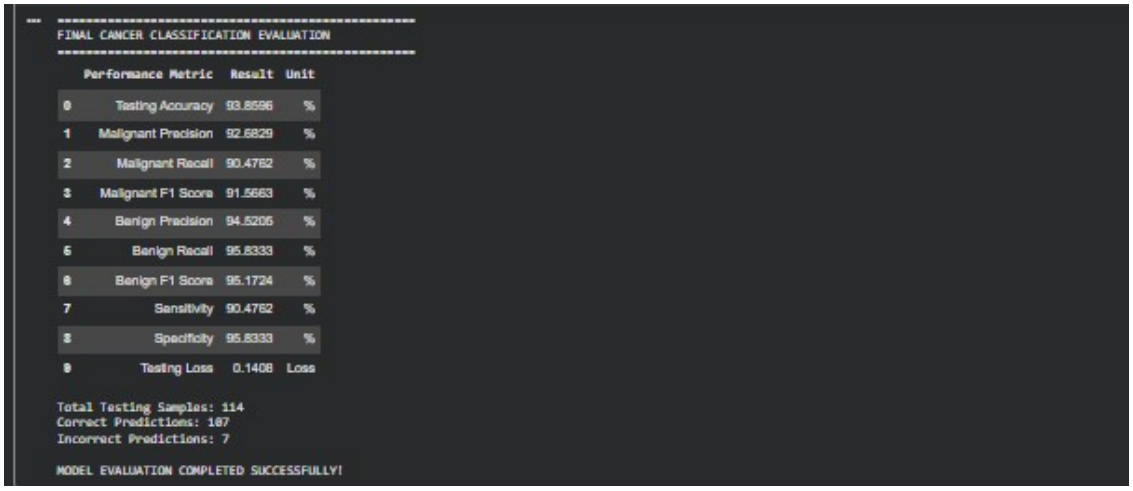


Figure 15. Final evaluation summary captured from the author's Google Colab session.

All training, validation and test metrics in this chapter refer to the same supplied Colab run. The 114-sample test set was kept separate from gradient updates and from fitting the final StandardScaler. The test results are a single-split estimate and are not evidence of clinical performance.

CONFUSION MATRIX AND ERROR ANALYSIS

5.5 Confusion Matrix

The supplied confusion matrix uses label order [0, 1]. Rows represent actual classes and columns represent predicted classes. Malignant is class 0 and Benign is class 1. The matrix is [[38, 4], [3, 69]]: 38 malignant samples were correctly classified, 4 malignant samples were classified as benign, 3 benign samples were classified as malignant, and 69 benign samples were correctly classified.

Actual \ Predicted	Malignant (0)	Benign (1)
Malignant (0)	38	4
Benign (1)	3	69

5.6 Error Interpretation

A false negative is a malignant-labelled sample classified as benign. A false positive is a benign-labelled sample classified as malignant. The two error types have different implications in a medical-themed classification example and should be reported separately. A high overall accuracy does not remove the need to inspect false negatives and class-wise recall.

5.7 Actual Confusion Matrix Output

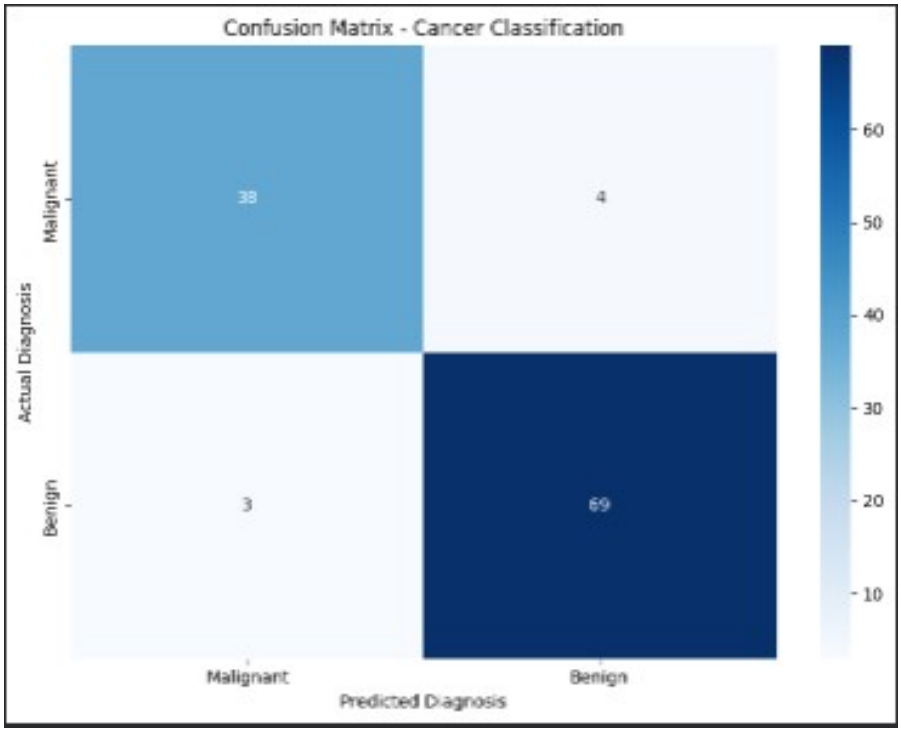


Figure 14. Confusion matrix captured from the author's Google Colab session for the 114-sample held-out test set.

Table 9. Class-wise classification report calculated from the supplied confusion-matrix counts.

Note: These class-wise values are calculated from the supplied confusion-matrix counts; they are not presented as a separate screenshot.

Class / Summary	Precision	Recall	F1-score	Support
Malignant	0.9268	0.9048	0.9157	42
Benign	0.9452	0.9583	0.9517	72
Accuracy	-	-	0.9386	114
Macro average	0.9360	0.9315	0.9337	114
Weighted average	0.9384	0.9386	0.9384	114

5.8 Result Discussion

The supplied test set contains 42 malignant and 72 benign samples. The model correctly classified 38 of the 42 malignant samples and 69 of the 72 benign samples. There were 4 false negatives and 3 false positives. Malignant-class recall (sensitivity) was 90.48%, while specificity for the benign class was 95.83%. The false-negative count is important to report explicitly, but this result does not establish clinical safety or diagnostic suitability.

TESTING AND VERIFICATION

5.9 Functional Test Cases

Test ID	Test scenario	Expected behaviour	Coverage
TC-01	Load dataset	569 samples and 30 features are available.	Documented
TC-02	Check missing values	Missing-value count is displayed.	Documented
TC-03	Check duplicate rows	Duplicate count is displayed.	Documented
TC-04	Split data	Stratified 80/20 split produces 455/114 samples.	Documented
TC-05	Scale features	Training scaler is fitted on fitting subset only.	Documented
TC-06	Build model	Input width 30 and output width 2.	Documented
TC-07	Train model	Training history is recorded for 10 epochs.	Documented
TC-08	Evaluate model	Test metrics and confusion matrix are generated.	Documented
TC-09	Valid app input	Thirty finite non-negative values are accepted.	Documented
TC-10	Invalid app input	Invalid or missing values return a validation message.	Documented
TC-11	Save/load model	Reloaded model reproduces predictions within tolerance.	Documented
TC-12	Save/load scaler	Reloaded scaler reproduces transformed values.	Documented

5.10 Verification Evidence

The supplied screenshots provide evidence for dataset structure, missing-value output, statistical analysis, class distribution, selected EDA charts, standardized data, model summary, training log and history, accuracy and loss curves, actual-versus-predicted results, confusion matrix, final evaluation summary and the running Gradio application with its displayed prediction output. The saved-model reload output is not represented by a screenshot in this report. The final notebook should be run from top to bottom after the required packages and Drive access are configured.

5.11 Reproducibility Checks

The saved model, scaler, feature order and class mapping should be treated as a matched set. A reloaded model should be tested against the original model on the same input samples. Any discrepancy beyond a small floating-point tolerance should be investigated before submission.

LIMITATIONS, ETHICS AND RESPONSIBLE USE

5.12 Dataset Limitations

- The dataset contains 569 samples and 30 features; this is small compared with the diversity of real-world clinical populations.
- The samples are from a specific historical dataset and may not represent current clinical practice or all demographic groups.
- The model receives precomputed numerical features and does not process original images.
- The model uses labels supplied by the dataset and does not independently validate their clinical correctness.
- No external test cohort, prospective study or clinical validation is performed.

5.13 Model Limitations

- The selected ANN is a compact educational architecture with one hidden layer.
- Performance can vary with random initialization, data split, library version and training configuration.
- A single train/test split provides only one estimate of generalization.
- Softmax probabilities are not necessarily calibrated probabilities of disease.
- Model output can be affected by feature measurement quality, units, ordering and out-of-distribution inputs.

5.14 Ethical and Safety Considerations

Cancer-related predictions are high-stakes. A model trained on a public benchmark must not be used to diagnose a person, determine treatment, reassure a patient or replace a qualified medical professional. The Gradio interface is intended only for demonstration using synthetic or dataset-derived educational values. No personal or identifiable patient information should be entered into a public demo link.

5.15 Responsible Communication

The project should be described as an educational binary-classification prototype. Claims such as “detects cancer accurately” or “diagnoses breast cancer” are too strong. Prefer wording such as “predicts the dataset class from 30 numerical features in an educational experiment.” The report must clearly state the held-out test performance once the actual values are verified and must also state that clinical validation was not performed.

CONCLUSION AND FUTURE SCOPE

6.1 Conclusion

The project demonstrates an end-to-end educational workflow for binary classification using the Breast Cancer Wisconsin (Diagnostic) dataset. The work includes loading a structured dataset, examining its quality and distributions, separating input features and labels, creating stratified data subsets, standardizing input measurements, building a feed-forward Artificial Neural Network, training the model, evaluating class-wise performance and creating an interactive prediction interface.

The implemented neural network uses 30 input features, one hidden layer of 20 ReLU neurons and two Softmax output neurons. It is configured with Adam optimization, sparse categorical cross-entropy and accuracy monitoring. The final training pipeline fits StandardScaler only on the fitting subset and reuses the same scaler during validation, testing and prediction.

The Gradio interface illustrates how a trained model can be connected to a user-facing form. It accepts the 30 numerical features, validates them, applies the stored preprocessing transformation and displays the model output. The project also preserves the model and supporting artefacts for reuse.

In the supplied Google Colab run, the held-out test accuracy was 93.86% and the test loss was 0.1408. Malignant-class precision was 92.68%, recall was 90.48% and F1-score was 91.57%. Benign-class precision was 94.52%, recall was 95.83% and F1-score was 95.17%. The confusion matrix contained 38 true positives, 4 false negatives, 3 false positives and 69 true negatives when Malignant was treated as the positive class. These values describe the supplied 114-sample test split only and do not establish clinical validity.

6.2 Future Scope

- Evaluate the model using repeated stratified cross-validation to obtain a more stable performance estimate.
- Compare the ANN with baseline models such as logistic regression, decision trees and support vector machines.
- Investigate regularization, dropout and early stopping using validation data only.
- Study class-wise error costs and evaluate thresholds without repeatedly using the final test set.
- Assess probability calibration and uncertainty communication.
- Test the model on an independent external dataset before considering any real-world use.
- Improve the interface with clearer units, input guidance and accessible layouts.
- Package the application with dependency pinning and reproducible deployment instructions.
- Add automated tests for preprocessing consistency, feature ordering and model reloading.

6.3 Final Statement

The project provides practical exposure to neural networks and the responsible interpretation of a machine-learning classifier. Its primary value is educational: understanding how data preparation, model design, evaluation and user-interface integration work together in a supervised-learning project.

REFERENCES

[1] UCI Machine Learning Repository. “Breast Cancer Wisconsin (Diagnostic) Data Set.” University of California, Irvine. https://archive.ics.uci.edu/dataset/17/breast+cancer+wisconsin+diagnostic (accessed 6 November 2026).

[2] Scikit-learn Developers. “load_breast_cancer.” Scikit-learn API Documentation. https://scikit-learn.org/stable/modules/generated/sklearn.datasets.load_breast_cancer.html (accessed 6 November 2026).

[3] Scikit-learn Developers. “StandardScaler.” Scikit-learn API Documentation. https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.StandardScaler.html (accessed 6 November 2026).

[4] Keras Team. “Save, Serialize, and Export Models.” Keras Developer Guide. https://keras.io/guides/serialization_and_saving/ (accessed 6 November 2026).

[5] Scikit-learn Developers. “Classification Metrics.” Scikit-learn User Guide and API documentation. https://scikit-learn.org/stable/modules/model_evaluation.html (accessed 6 November 2026).

[6] Project specification supplied for “Cancer Classification Using Neural Networks,” September 2026.

[7] Reference notebook supplied for “Breast Cancer Classification with Neural Networks,” September 2026.

[8] Gradio Team. “Blocks.” Gradio Documentation. https://www.gradio.app/docs/gradio/blocks (accessed 6 November 2026).

[9] W. N. Street, W. H. Wolberg, and O. L. Mangasarian. “Nuclear feature extraction for breast tumor diagnosis.” Proceedings / Electronic Imaging, 1993. Dataset provenance and associated research reference as listed by UCI.

APPENDIX A - FEATURE INVENTORY

The following table lists the 30 feature names used by Scikit-learn in their original order. The same order must be preserved when preparing model inputs and using the saved feature_names.json file.

No.	Feature name	Group
1	mean radius	Mean
2	mean texture	Mean
3	mean perimeter	Mean
4	mean area	Mean
5	mean smoothness	Mean
6	mean compactness	Mean
7	mean concavity	Mean
8	mean concave points	Mean
9	mean symmetry	Mean
10	mean fractal dimension	Mean
11	radius error	Standard error
12	texture error	Standard error
13	perimeter error	Standard error
14	area error	Standard error
15	smoothness error	Standard error
16	compactness error	Standard error
17	concavity error	Standard error
18	concave points error	Standard error
19	symmetry error	Standard error
20	fractal dimension error	Standard error
21	worst radius	Worst
22	worst texture	Worst
23	worst perimeter	Worst
24	worst area	Worst
25	worst smoothness	Worst
26	worst compactness	Worst
27	worst concavity	Worst
28	worst concave points	Worst
29	worst symmetry	Worst
30	worst fractal dimension	Worst

APPENDIX B - PROJECT RELEASE AND REPRODUCIBILITY NOTES

B.1 Saved Artefacts

Folder / file	Purpose	Final check
models/cancer_model.keras	Trained ANN model	Included in project structure
models/training_scaler.joblib	Fitted preprocessing scaler	Included in project structure
metadata/feature_names.json	Original feature order	Included in project structure
metadata/class_mapping.json	Target label names	Included in project structure
metadata/model_metadata.json	Model and training configuration	Included in project structure
results/training_history.csv	Epoch-wise training history	Included in project structure
results/evaluation_summary.csv	Evaluation metrics	Included in project structure
results/confusion_matrix.csv	Confusion matrix values	Included in project structure
results/classification_report.txt	Class-wise evaluation report	Included in project structure

B.2 Final Submission Checklist

- Prepared: the editable project report (.docx) and corresponding PDF are available.
- Prepared: the cover page displays the project title and author name, SANKET KISHOR SUTAR.
- Prepared: the report includes the chapters specified in the September 2026 project guide.
- Prepared: supplied screenshots document dataset inspection, EDA, model architecture, training, accuracy and loss, predictions and confusion-matrix results.
- Pending: prepare a 10–15 slide PowerPoint covering the problem statement, objectives, dataset, methodology, model architecture, results and conclusion.
- Pending: export the final Jupyter Notebook (.ipynb) and organize the required Python source files (.py) for the complete project package.
- Pending: prepare README.md with the project overview, setup, execution instructions, model details, results and limitations.
- Pending: prepare requirements.txt using the software versions verified in the final execution environment.
- Pending: publish and verify the complete GitHub repository, then record its repository URL.
- Pending: email the complete submission to info@navitechsolutions.com after the GitHub repository is published, including the repository URL and required attachments.

B.3 Output Provenance and Evidence Notes

The dataset counts, class distribution, DataFrame inspection, missing-value output, descriptive statistics, EDA charts, model summary, training log, training-history table, learning curves, actual-versus-predicted table, confusion matrix and final evaluation summary in this report were transcribed or cropped from the Google Colab screenshots supplied by SANKET KISHOR SUTAR. The actual Gradio screenshots in Figure 9 were also supplied by the author and show the live measurement form, prediction control and a demonstration output. The class-wise classification report table is calculated from the supplied confusion-matrix counts. The full correlation heatmap and ANN/system architecture diagrams are report figures generated from the documented dataset and model design. A saved-model reload screenshot was not supplied.